

```

using System;
using System.IO;

namespace BibliotecaUniversitaria
{
    class Program
    {
        // =====
        // STRUCTS REQUERIDOS (Sección 4.2)
        // =====
        struct Libro
        {
            public string codigo;
            public string titulo;
            public string autor;
            public string editorial;
            public int anioPublicacion;
            public string categoria;
            public int cantidadEjemplares;
        }

        struct Usuario
        {
            public string carne;
            public string nombreCompleto;
            public string carrera;
            public string correoElectronico;
            public string telefono;
            public string estado; // Activo/Inactivo
        }

        struct Prestamo
        {
            public string carneUsuario;
            public string codigoLibro;
            public string fechaPrestamo;
            public string fechaEstimadaDevolucion;
            public string estado; // Activo/Devuelto
        }

        // =====
        // ARREGLOS ESTÁTICOS CON LÍMITES ESTRUCTOS (Sección 4.2)
        // =====
        static Libro[] libros = new Libro[10];
        static Usuario[] usuarios = new Usuario[5];
        static Prestamo[] prestamos = new Prestamo[10];

        // Contadores manuales para controlar el llenado de los vectores
    }
}

```

```

static int contadorLibros = 0;
static int contadorUsuarios = 0;
static int contadorPrestamos = 0;

// Rutas de archivos (Sección 4.4)
static string rutaCarpeta = "Data";
static string archivoLibros = "Data/libros.csv";
static string archivoUsuarios = "Data/usuarios.txt";
static string archivoPrestamos = "Data/prestamos.txt";

static void Main(string[] args)
{
    // Crear la carpeta si no existe antes de cargar nada
    if (!Directory.Exists(rutaCarpeta))
    {
        Directory.CreateDirectory(rutaCarpeta);
    }

    // Cargar los datos guardados al iniciar el programa
    CargarDatosDesdeArchivos();

    int opcionPrincipal = 0;

    do
    {
        Console.Clear();
        Console.WriteLine("=====");
        Console.WriteLine(" SISTEMA INTEGRAL DE GESTIÓN DE BIBLIOTECA ");
        Console.WriteLine("=====");
        Console.WriteLine("1. Gestión de Libros");
        Console.WriteLine("2. Gestión de Usuarios");
        Console.WriteLine("3. Gestión de Préstamos");
        Console.WriteLine("4. Salir del Sistema");
        Console.WriteLine("=====");
        Console.Write("Seleccione una opción (1-4): ");

        try
        {
            opcionPrincipal = int.Parse(Console.ReadLine());

            switch (opcionPrincipal)
            {
                case 1:
                    MenuLibros();
                    break;
                case 2:
                    MenuUsuarios();
                    break;
            }
        }
    } while (opcionPrincipal != 4);
}

```

```

        case 3:
            MenuPrestamos();
            break;
        case 4:
            Console.WriteLine("\nGuardando datos en el disco... ¡Feliz día!");
            GuardarDatosEnArchivos();
            Console.ReadKey();
            break;
        default:
            Console.WriteLine("Opción inválida. Intente de nuevo.");
            Console.ReadKey();
            break;
    }
}
catch (FormatException)
{
    Console.WriteLine("\n[Error de Formato]: Debe ingresar un número entero.");
    Console.ReadKey();
}

} while (opcionPrincipal != 4);
}

// =====
// SUBMENÚS (Sección 4.1)
// =====
static void MenuLibros()
{
    int op = 0;
    do
    {
        Console.Clear();
        Console.WriteLine("--- MÓDULO A: GESTIÓN DE LIBROS ---");
        Console.WriteLine("1. Registrar nuevo libro");
        Console.WriteLine("2. Buscar libro por código");
        Console.WriteLine("3. Listado completo de libros");
        Console.WriteLine("4. Eliminar un libro");
        Console.WriteLine("5. Volver al menú principal");
        Console.Write("Seleccione una opción: ");
        try
        {
            op = int.Parse(Console.ReadLine());
            switch (op)
            {
                case 1: RegistrarLibro(); break;
                case 2: BuscarLibro(); break;
                case 3: ListarLibros(); break;
                case 4: EliminarLibro(); break;
            }
        }
    }
}

```

```

        }
    }
    catch (FormatException) { Console.WriteLine("Entrada no válida.");
Console.ReadKey(); }
    } while (op != 5);
}

static void MenuUsuarios()
{
    int op = 0;
    do
    {
        Console.Clear();
        Console.WriteLine("--- MÓDULO B: GESTIÓN DE USUARIOS ---");
        Console.WriteLine("1. Registrar nuevo usuario");
        Console.WriteLine("2. Buscar usuario por carné o nombre");
        Console.WriteLine("3. Listado completo de usuarios");
        Console.WriteLine("4. Volver al menú principal");
        Console.Write("Seleccione una opción: ");
        try
        {
            op = int.Parse(Console.ReadLine());
            switch (op)
            {
                case 1: RegistrarUsuario(); break;
                case 2: BuscarUsuario(); break;
                case 3: ListarUsuarios(); break;
            }
        }
        catch (FormatException) { Console.WriteLine("Entrada no válida.");
Console.ReadKey(); }
    } while (op != 4);
}

static void MenuPrestamos()
{
    int op = 0;
    do
    {
        Console.Clear();
        Console.WriteLine("--- MÓDULO C: GESTIÓN DE PRÉSTAMOS ---");
        Console.WriteLine("1. Registrar préstamo");
        Console.WriteLine("2. Registrar devolución");
        Console.WriteLine("3. Ver préstamos activos de un usuario");
        Console.WriteLine("4. Actualizar estado de préstamo");
        Console.WriteLine("5. Volver al menú principal");
        Console.Write("Seleccione una opción: ");
        try

```

```

    {
        op = int.Parse(Console.ReadLine());
        switch (op)
        {
            case 1: RegistrarPrestamo(); break;
            case 2: RegistrarDevolucion(); break;
            case 3: HistorialPrestamosUsuario(); break;
            case 4: ActualizarEstadoPrestamo(); break;
        }
    }
    catch (FormatException) { Console.WriteLine("Entrada no válida.");
Console.ReadKey(); }
    } while (op != 5);
}

// =====
// LÓGICA MÓDULO A: LIBROS (Sección 3.1)
// =====
static void RegistrarLibro()
{
    Console.Clear();
    Console.WriteLine("== REGISTRAR LIBRO ==");

    if (contadorLibros >= 10)
    {
        Console.WriteLine("Error: Se ha alcanzado el límite máximo de 10 libros en
memoria.");
        Console.ReadKey();
        return;
    }

    Libro nuevoLibro = new Libro();

    // Validación de Código Alfanumérico de 8 caracteres
    while (true)
    {
        Console.Write("Ingrese código (8 caracteres alfanuméricos): ");
        string cod = Console.ReadLine().Trim();
        if (cod.Length == 8)
        {
            // Verificar si ya existe
            bool existe = false;
            for (int i = 0; i < contadorLibros; i++)
            {
                if (libros[i].codigo == cod) existe = true;
            }

            if (!existe)

```

```

        {
            nuevoLibro.codigo = cod;
            break;
        }
        else Console.WriteLine("Ese código de libro ya está registrado.");
    }
    else Console.WriteLine("El código debe tener exactamente 8 caracteres.");
}

```

```

Console.Write("Título: ");
nuevoLibro.titulo = Console.ReadLine().Trim();
Console.Write("Autor: ");
nuevoLibro.autor = Console.ReadLine().Trim();
Console.Write("Editorial: ");
nuevoLibro.editorial = Console.ReadLine().Trim();

```

```

// Validación Año Publicación
while (true)
{
    Console.Write("Año de publicación (1900 - 2026): ");
    try
    {
        int anio = int.Parse(Console.ReadLine());
        if (anio >= 1900 && anio <= 2026)
        {
            nuevoLibro.anioPublicacion = anio;
            break;
        }
        Console.WriteLine("Año fuera del rango permitido.");
    }
    catch { Console.WriteLine("Año inválido."); }
}

```

```

Console.Write("Categoría: ");
nuevoLibro.categoria = Console.ReadLine().Trim();

```

```

// Validación Cantidad Ejemplares
while (true)
{
    Console.Write("Cantidad de ejemplares disponibles: ");
    try
    {
        int cant = int.Parse(Console.ReadLine());
        if (cant >= 0)
        {
            nuevoLibro.cantidadEjemplares = cant;
            break;
        }
    }
}

```

```

        Console.WriteLine("La cantidad no puede ser negativa.");
    }
    catch { Console.WriteLine("Cantidad inválida."); }
}

libros[contadorLibros] = nuevoLibro;
contadorLibros++;
Console.WriteLine("\nLibro guardado con éxito exitosamente en memoria!");
Console.ReadKey();
}

static void BuscarLibro()
{
    Console.Clear();
    Console.WriteLine("== BUSCAR LIBRO BY CÓDIGO ==");
    Console.Write("Ingrese el código a buscar: ");
    string buscado = Console.ReadLine().Trim();

    bool encontrado = false;
    for (int i = 0; i < contadorLibros; i++)
    {
        if (libros[i].codigo.ToLower() == buscado.ToLower())
        {
            Console.WriteLine($"nCódigo: {libros[i].codigo}");
            Console.WriteLine($"Título: {libros[i].titulo}");
            Console.WriteLine($"Autor: {libros[i].autor}");
            Console.WriteLine($"Editorial: {libros[i].editorial}");
            Console.WriteLine($"Año: {libros[i].anioPublicacion}");
            Console.WriteLine($"Categoría: {libros[i].categoria}");
            Console.WriteLine($"Disponibles: {libros[i].cantidadEjemplares}");
            encontrado = true;
            break;
        }
    }

    if (!encontrado) Console.WriteLine("Libro no encontrado en el sistema.");
    Console.ReadKey();
}

static void ListarLibros()
{
    Console.Clear();
    Console.WriteLine("== LISTA DE LIBROS REGISTRADOS ==");
    if (contadorLibros == 0) Console.WriteLine("No hay libros en memoria.");

    for (int i = 0; i < contadorLibros; i++)
    {

```

```

        Console.WriteLine($"{i + 1}. [{libros[i].codigo}] {libros[i].titulo} - {libros[i].autor}
(Dispo: {libros[i].cantidadEjemplares})");
    }
    Console.ReadKey();
}

```

```

static void EliminarLibro()
{
    Console.Clear();
    Console.WriteLine("== ELIMINAR LIBRO ==");
    Console.Write("Ingrese el código del libro a borrar: ");
    string cod = Console.ReadLine().Trim();

    int indiceEncontrado = -1;
    for (int i = 0; i < contadorLibros; i++)
    {
        if (libros[i].codigo.ToLower() == cod.ToLower())
        {
            indiceEncontrado = i;
            break;
        }
    }

    if (indiceEncontrado != -1)
    {
        // Mover los elementos hacia atrás para cubrir el hueco (Lógica de vectores
        tradicional)
        for (int i = indiceEncontrado; i < contadorLibros - 1; i++)
        {
            libros[i] = libros[i + 1];
        }
        contadorLibros--;
        Console.WriteLine("Libro removido del registro.");
    }
    else Console.WriteLine("El libro especificado no existe.");

    Console.ReadKey();
}

```

```

// =====
// LÓGICA MÓDULO B: USUARIOS (Sección 3.1)
// =====
static void RegistrarUsuario()
{
    Console.Clear();
    Console.WriteLine("== REGISTRAR USUARIO ==");

    if (contadorUsuarios >= 5)

```



```

{
    Console.WriteLine("Error: Límite máximo de 5 usuarios alcanzado.");
    Console.ReadKey();
    return;
}

Usuario nuevoUsuario = new Usuario();

// Validación carné de 8 dígitos numéricos
while (true)
{
    Console.Write("Carné (8 dígitos numéricos): ");
    string car = Console.ReadLine().Trim();

    bool esNumerico = true;
    foreach (char c in car)
    {
        if (!char.IsDigit(c)) esNumerico = false;
    }

    if (car.Length == 8 && esNumerico)
    {
        nuevoUsuario.carne = car;
        break;
    }
    Console.WriteLine("Formato erróneo. Deben ser exactamente 8 números.");
}

Console.Write("Nombre Completo: ");
nuevoUsuario.nombreCompleto = Console.ReadLine().Trim();
Console.Write("Carrera: ");
nuevoUsuario.carrera = Console.ReadLine().Trim();

// Validación de correo con '@' y '.' posterior
while (true)
{
    Console.Write("Correo Electrónico: ");
    string mail = Console.ReadLine().Trim();
    int arrobaldx = mail.IndexOf('@');
    int puntoldx = mail.LastIndexOf('.');

    if (arrobaldx != -1 && puntoldx != -1 && puntoldx > arrobaldx)
    {
        nuevoUsuario.correoElectronico = mail;
        break;
    }
    Console.WriteLine("Correo inválido. Debe contener '@' y un '.' después del
mismo.");
}

```

```

    }

    Console.WriteLine("Teléfono: ");
    nuevoUsuario.telefono = Console.ReadLine().Trim();
    nuevoUsuario.estado = "Activo"; // Por defecto al registrarse

    usuarios[contadorUsuarios] = nuevoUsuario;
    contadorUsuarios++;
    Console.WriteLine("Usuario registrado con éxito.");
    Console.ReadKey();
}

static void BuscarUsuario()
{
    Console.Clear();
    Console.WriteLine("== BUSCAR USUARIO ==");
    Console.WriteLine("Ingrese Carné o Nombre a buscar: ");
    string filtro = Console.ReadLine().Trim().ToLower();

    bool hallado = false;
    for (int i = 0; i < contadorUsuarios; i++)
    {
        if (usuarios[i].carne.ToLower() == filtro ||
            usuarios[i].nombreCompleto.ToLower().Contains(filtro))
        {
            Console.WriteLine($"Carné: {usuarios[i].carne}");
            Console.WriteLine($"Nombre: {usuarios[i].nombreCompleto}");
            Console.WriteLine($"Carrera: {usuarios[i].carrera}");
            Console.WriteLine($"Correo: {usuarios[i].correoElectronico}");
            Console.WriteLine($"Estado: {usuarios[i].estado}");
            hallado = true;
        }
    }

    if (!hallado) Console.WriteLine("No se encontraron coincidencias.");
    Console.ReadKey();
}

static void ListarUsuarios()
{
    Console.Clear();
    Console.WriteLine("== LISTA TOTAL DE USUARIOS ==");
    if (contadorUsuarios == 0) Console.WriteLine("No hay usuarios registrados.");

    for (int i = 0; i < contadorUsuarios; i++)
    {
        Console.WriteLine($"- {usuarios[i].carne} | {usuarios[i].nombreCompleto} |
{usuarios[i].carrera} ({usuarios[i].estado})");
    }
}

```

```

    }
    Console.ReadKey();
}

// =====
// LÓGICA MÓDULO C: PRÉSTAMOS (Sección 3.1)
// =====
static void RegistrarPrestamo()
{
    Console.Clear();
    Console.WriteLine("== REGISTRAR PRÉSTAMO ==");

    if (contadorPrestamos >= 10)
    {
        Console.WriteLine("Límite de 10 préstamos simultáneos lleno.");
        Console.ReadKey();
        return;
    }

    Console.Write("Ingrese Carné del usuario: ");
    string carne = Console.ReadLine().Trim();
    int userIdx = -1;
    for (int i = 0; i < contadorUsuarios; i++)
    {
        if (usuarios[i].carne == carne) userIdx = i;
    }

    if (userIdx == -1 || usuarios[userIdx].estado == "Inactivo")
    {
        Console.WriteLine("Usuario no existe o está Inactivo.");
        Console.ReadKey();
        return;
    }

    Console.Write("Ingrese Código del Libro: ");
    string cod = Console.ReadLine().Trim();
    int libroIdx = -1;
    for (int i = 0; i < contadorLibros; i++)
    {
        if (libros[i].codigo == cod) libroIdx = i;
    }

    if (libroIdx == -1)
    {
        Console.WriteLine("El libro especificado no existe.");
        Console.ReadKey();
        return;
    }
}

```

```

if (libros[libroIdx].cantidadEjemplares <= 0)
{
    Console.WriteLine("No hay existencias disponibles de este libro actualmente.");
    Console.ReadKey();
    return;
}

Prestamo nuevoP = new Prestamo();
nuevoP.carneUsuario = carne;
nuevoP.codigoLibro = cod;

// Validación básica de fechas por longitud y separadores (ej: dd/mm/yyyy)
while (true)
{
    Console.WriteLine("Fecha del Préstamo (dd/mm/yyyy): ");
    string f1 = Console.ReadLine().Trim();
    if (f1.Length == 10 && f1[2] == '/' && f1[5] == '/')
    {
        nuevoP.fechaPrestamo = f1;
        break;
    }
    Console.WriteLine("Formato de fecha inválido.");
}

while (true)
{
    Console.WriteLine("Fecha Estimada de Devolución (dd/mm/yyyy): ");
    string f2 = Console.ReadLine().Trim();
    if (f2.Length == 10 && f2[2] == '/' && f2[5] == '/')
    {
        nuevoP.fechaEstimadaDevolucion = f2;
        break;
    }
    Console.WriteLine("Formato de fecha inválido.");
}

nuevoP.estado = "Activo";

// Modificar inventario en memoria
libros[libroIdx].cantidadEjemplares--;

prestamos[contadorPrestamos] = nuevoP;
contadorPrestamos++;

Console.WriteLine("¡Préstamo concedido con éxito!");
Console.ReadKey();
}

```

```

static void RegistrarDevolucion()
{
    Console.Clear();
    Console.WriteLine("== CONTROL DE DEVOLUCIONES ==");
    Console.Write("Ingrese Carné del usuario: ");
    string carne = Console.ReadLine().Trim();
    Console.Write("Ingrese Código del libro: ");
    string libro = Console.ReadLine().Trim();

    bool hecho = false;
    for (int i = 0; i < contadorPrestamos; i++)
    {
        if (prestamos[i].carneUsuario == carne && prestamos[i].codigoLibro == libro &&
prestamos[i].estado == "Activo")
        {
            prestamos[i].estado = "Devuelto";

            // Devolver al stock
            for (int j = 0; j < contadorLibros; j++)
            {
                if (libros[j].codigo == libro) libros[j].cantidadEjemplares++;
            }

            Console.WriteLine("Devolución asentada y stock actualizado.");
            hecho = true;
            break;
        }
    }

    if (!hecho) Console.WriteLine("No se encontró ningún préstamo activo con esas
credenciales.");
    Console.ReadKey();
}

static void HistorialPrestamosUsuario()
{
    Console.Clear();
    Console.WriteLine("== PRÉSTAMOS ACTIVOS POR USUARIO ==");
    Console.Write("Ingrese Carné del usuario: ");
    string carne = Console.ReadLine().Trim();

    int cont = 0;
    for (int i = 0; i < contadorPrestamos; i++)
    {
        if (prestamos[i].carneUsuario == carne && prestamos[i].estado == "Activo")
        {

```

```

        Console.WriteLine($"- Libro: {prestamos[i].codigoLibro} | Prestado el:
{prestamos[i].fechaPrestamo} | Devuelve el: {prestamos[i].fechaEstimadaDevolucion}");
        cont++;
    }
}

```

```

    if (cont == 0) Console.WriteLine("El usuario no posee préstamos activos.");
    Console.ReadKey();
}

```

```

static void ActualizarEstadoPrestamo()

```

```

{
    Console.Clear();
    Console.WriteLine("== MODIFICAR ESTADO DEL PRÉSTAMO ==");
    Console.Write("Ingrese Carné del usuario: ");
    string carne = Console.ReadLine().Trim();
    Console.Write("Ingrese Código del libro: ");
    string libro = Console.ReadLine().Trim();

    for (int i = 0; i < contadorPrestamos; i++)
    {
        if (prestamos[i].carneUsuario == carne && prestamos[i].codigoLibro == libro)
        {
            Console.WriteLine($"Estado actual: {prestamos[i].estado}");
            Console.Write("Ingrese nuevo estado (Activo/Devuelto/Vencido): ");
            prestamos[i].estado = Console.ReadLine().Trim();
            Console.WriteLine("Estado cambiado exitosamente.");
            Console.ReadKey();
            return;
        }
    }
    Console.WriteLine("Préstamo no localizado.");
    Console.ReadKey();
}

```

```

// =====
// MANEJO DE ARCHIVOS: PERSISTENCIA (Sección 4.4)
// =====

```

```

static void GuardarDatosEnArchivos()

```

```

{
    try
    {
        // 1. Guardar Libros (Formato CSV)
        using (StreamWriter sw = new StreamWriter(archivoLibros, false))
        {
            for (int i = 0; i < contadorLibros; i++)
            {

```

```

sw.WriteLine($"{libros[i].codigo},{libros[i].titulo},{libros[i].autor},{libros[i].editorial},{libros[i].anio
Publicacion},{libros[i].categoria},{libros[i].cantidadEjemplares}");
    }
}

```

```

// 2. Guardar Usuarios (Formato TXT separado por pipelines '|')
using (StreamWriter sw = new StreamWriter(archivoUsuarios, false))
{
    for (int i = 0; i < contadorUsuarios; i++)
    {

```

```

sw.WriteLine($"{usuarios[i].carne}|{usuarios[i].nombreCompleto}|{usuarios[i].carrera}|{usuari
os[i].correoElectronico}|{usuarios[i].telefono}|{usuarios[i].estado}");
    }
}

```

```

// 3. Guardar Préstamos (Formato TXT separado por pipelines '|')
using (StreamWriter sw = new StreamWriter(archivoPrestamos, false))
{
    for (int i = 0; i < contadorPrestamos; i++)
    {

```

```

sw.WriteLine($"{prestamos[i].carneUsuario}|{prestamos[i].codigoLibro}|{prestamos[i].fechaPr
estamo}|{prestamos[i].fechaEstimadaDevolucion}|{prestamos[i].estado}");
    }
}

```

```

    Console.WriteLine("Datos respaldados correctamente en la carpeta Data/.");
}
catch (Exception ex)
{
    Console.WriteLine("Error al guardar: " + ex.Message);
}
}

```

```

static void CargarDatosDesdeArchivos()
{
    try
    {
        // Cargar Libros
        if (File.Exists(archivoLibros))
        {
            string[] lineas = File.ReadAllLines(archivoLibros);
            contadorLibros = 0;
            foreach (string l in lineas)
            {
                if (string.IsNullOrEmpty(l)) continue;

```

```

string[] datos = l.Split(',');
if (datos.Length == 7 && contadorLibros < 10)
{
    libros[contadorLibros].codigo = datos[0];
    libros[contadorLibros].titulo = datos[1];
    libros[contadorLibros].autor = datos[2];
    libros[contadorLibros].editorial = datos[3];
    libros[contadorLibros].anioPublicacion = int.Parse(datos[4]);
    libros[contadorLibros].categoria = datos[5];
    libros[contadorLibros].cantidadEjemplares = int.Parse(datos[6]);
    contadorLibros++;
}
}
}

```

```

// Cargar Usuarios
if (File.Exists(archivoUsuarios))
{
    string[] lineas = File.ReadAllLines(archivoUsuarios);
    contadorUsuarios = 0;
    foreach (string l in lineas)
    {
        if (string.IsNullOrEmpty(l)) continue;
        string[] datos = l.Split('|');
        if (datos.Length == 6 && contadorUsuarios < 5)
        {
            usuarios[contadorUsuarios].carne = datos[0];
            usuarios[contadorUsuarios].nombreCompleto = datos[1];
            usuarios[contadorUsuarios].carrera = datos[2];
            usuarios[contadorUsuarios].correoElectronico = datos[3];
            usuarios[contadorUsuarios].telefono = datos[4];
            usuarios[contadorUsuarios].estado = datos[5];
            contadorUsuarios++;
        }
    }
}

```

```

// Cargar Préstamos
if (File.Exists(archivoPrestamos))
{
    string[] lineas = File.ReadAllLines(archivoPrestamos);
    contadorPrestamos = 0;
    foreach (string l in lineas)
    {
        if (string.IsNullOrEmpty(l)) continue;
        string[] datos = l.Split('|');
        if (datos.Length == 5 && contadorPrestamos < 10)
        {

```



```

        prestamos[contadorPrestamos].carneUsuario = datos[0];
        prestamos[contadorPrestamos].codigoLibro = datos[1];
        prestamos[contadorPrestamos].fechaPrestamo = datos[2];
        prestamos[contadorPrestamos].fechaEstimadaDevolucion = datos[3];
        prestamos[contadorPrestamos].estado = datos[4];
        contadorPrestamos++;
    }
}
}
}
catch
{
    // Si da error de parseo al leer archivos vacíos o corruptos, inicia en limpio
    silenciosamente
}
}
}
}

```